

Table des matières

1	Présentation du projet	2
1.1	Contexte	2
1.2	Ce que Nova AI n'est PAS	2
1.3	L'idée fondatrice : AI Skill Graph	2
1.4	Différenciation réelle (fonctionnalités impossibles avec ChatGPT/Claude seuls)	3
2	Stack technique (validée, stable, réaliste en 2 mois)	4
3	Fonctionnalités du MVP (périmètre réellement livré en 2 mois)	5
3.1	Authentification et profil	5
3.2	Génération de roadmap	5
3.3	Assistant IA (chat)	5
3.4	Gestion des tâches et projets	5
3.5	Documents et RAG	5
3.6	Exercices générés + répétition espacée * (différenciant)	5
3.7	AI Mentor Engine * (différenciant)	5
3.8	Certificat vérifiable * (différenciant)	6
3.9	Recherche cross-données * (différenciant)	6
3.10	Dashboard de progression	6
3.11	Historique	6
3.12	Automatisation n8n	6
4	Modèle de données (schéma PostgreSQL)	6
4.1	Champ clé pour la répétition espacée	7
4.2	Où et comment PostgreSQL est utilisé	8
5	Architecture technique	8
6	Vision future (non développée dans le MVP – présentée comme roadmap en soutenance)	9
7	Exigences non fonctionnelles	9
7.1	Sécurité et confidentialité	9
7.2	Performance	9
7.3	Scalabilité	9
8	Répartition de l'équipe (2 personnes)	9
8.1	Répartition détaillée des tâches	10
9	Planning indicatif (8 semaines)	11
10	Livrables attendus	12
11	Indicateurs de succès (KPIs) pour la soutenance	12
12	Points ouverts à valider	12

1 Présentation du projet

1.1 Contexte

Nova AI est un mentor intelligent qui accompagne les étudiants et professionnels dans leur apprentissage et leur organisation quotidienne. Contrairement à un chatbot généraliste, Nova AI **construit un profil, planifie un parcours personnalisé, suit la progression dans le temps**, et **agit de façon autonome** (relances, replanification, génération de certificats) sans attendre systématiquement une demande explicite de l'utilisateur.

1.2 Ce que Nova AI n'est PAS

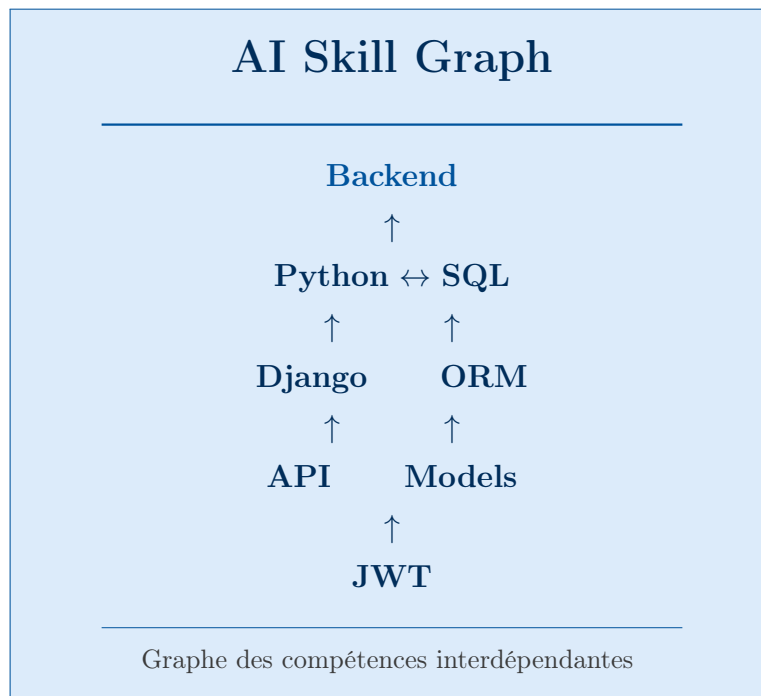
Pour éviter toute ambiguïté en soutenance :

- Ce n'est pas un remplaçant de ChatGPT/Claude – Nova AI **utilise** un LLM (Llama 3 via Ollama) comme moteur de génération de texte, exactement comme n'importe quelle application IA.
- La valeur ajoutée de Nova AI n'est **pas** dans le texte généré par le LLM, mais dans le **système autour** : mémoire structurée, actions autonomes, calculs algorithmiques, objets vérifiables.

1.3 L'idée fondatrice : AI Skill Graph

Inspiration issue des premières réflexions du projet.

Imaginez un système où chaque compétence est représentée sous forme de graphe.



Ce graphe de compétences est le cœur de la personnalisation. Le système ne se contente pas de suivre un parcours linéaire ; il comprend les interdépendances entre les sujets.

Pourquoi cette idée est puissante ? Parce que ce n'est pas ChatGPT. Ce n'est pas Claude. C'est **votre algorithme**. Le LLM ne décide pas seul. Votre backend, grâce à ce graphe, possède une vue d'ensemble et prend des décisions éclairées.

Un système adaptatif. Par exemple, si un étudiant échoue à un exercice sur **JWT**, le système comprend automatiquement que **ORM** est encore faible et ne peut pas être une compétence prérequis. Il change automatiquement le parcours pour proposer des exercices de renforcement sur l'ORM avant de réessayer JWT.

Un suivi continu. Quand l'étudiant termine un exercice, le système ne met pas seulement un score. Il met à jour les compétences dans le graphe, par exemple :

Python	100%
SQL	80%
JWT	20%
Mise à jour automatique	

Puis, l'IA comprend automatiquement la nouvelle situation et ajuste le planning et les recommandations.

1.4 Différenciation réelle (fonctionnalités impossibles avec ChatGPT/Claude seuls)

#	Fonctionnalité	Pourquoi ChatGPT/Claude ne peuvent pas le faire seuls
1	AI Mentor Engine – relance automatique en cas d'inactivité	ChatGPT/Claude ne s'exécutent que sur demande ; ils n'ont pas de processus qui tourne en arrière-plan entre les sessions
2	Répétition espacée (spaced repetition) – replanification automatique des révisions selon un algorithme (type Anki)	C'est un calcul algorithmique stocké en base et exécuté par le backend, pas une génération de texte
3	Certificat vérifiable – génération d'un PDF avec QR code, numéro unique, et route API de vérification	ChatGPT peut écrire le texte mais ne peut pas créer un objet système persistant et vérifiable par un tiers
4	Recherche structurée cross-données – retrouver en une requête tout ce qui est lié à un sujet (documents + exercices + notes + projets)	Nécessite des tables reliées en base de données ; ChatGPT ne peut chercher que dans le texte d'une conversation

#	Fonctionnalité	Pourquoi ChatGPT/Claude ne peuvent pas le faire seuls
5	Multi-utilisateurs avec vue admin – statistiques agrégées, taux de réussite global	ChatGPT est un outil 1-à-1 ; il n’a aucune notion d’ensemble d’utilisateurs

Ces 5 points sont ceux mis en avant dans la soutenance comme différenciation technique réelle – pas des reformulations de capacités déjà présentes dans un LLM générique.

2 Stack technique (validée, stable, réaliste en 2 mois)

Couche	Technologie	Rôle
Frontend	React.js + Tailwind CSS	Interface utilisateur (dashboard, chat, documents)
Backend	Django + Django REST Framework	API REST, logique métier, authentification, tâches planifiées
Base de données	PostgreSQL + extension pgvector	Stockage relationnel + recherche vectorielle (RAG)
IA	Ollama (Llama 3 ou Mistral)	Chat, génération de roadmap/exercices, résumé
Automatisation	n8n	Emails, rappels, rapports, déclenchement de l’AI Mentor Engine
Génération PDF/QR	Bibliothèque Python (ex. reportlab + qrcode)	Génération des certificats vérifiables

Aucun changement de stack par rapport aux versions précédentes – pas d’Angular, LangChain, Qdrant, Celery/Redis, Whisper : ces briques restent en section 7 (Vision future).

3 Fonctionnalités du MVP (périmètre réellement livré en 2 mois)

3.1 Authentification et profil

- Inscription / connexion / déconnexion
- **Onboarding intelligent** : court entretien guidé à l'inscription (objectif, niveau, disponibilité hebdomadaire, langue préférée)

3.2 Génération de roadmap

- À partir du profil, génération automatique d'un parcours (étapes ordonnées) et d'un planning indicatif
- Modifiable manuellement par l'utilisateur

3.3 Assistant IA (chat)

Expliquer un concept, résumer, corriger un texte, traduire, répondre aux questions générales.

3.4 Gestion des tâches et projets

CRUD complet (créer/modifier/supprimer), priorité, statut, échéance, regroupement en projets.

3.5 Documents et RAG

- Upload PDF/Word/TXT
- Bouton "Analyser" → questions sur le contenu d'un document précis (voir spécification frontend déjà définie)
- Stockage des embeddings dans PostgreSQL via pgvector

3.6 Exercices générés + répétition espacée * (différenciant)

- Génération de quelques questions (QCM ou ouvertes) liées à une étape du parcours
- Correction automatique avec feedback généré par le LLM
- **Algorithme de répétition espacée** : selon le score obtenu, calcul automatique de la date à laquelle l'exercice (ou un exercice similaire) doit être représenté à l'utilisateur, et insertion automatique dans son planning

3.7 AI Mentor Engine * (différenciant)

- Détection d'inactivité (pas de connexion depuis X jours) → déclenchement d'un email de relance via n8n
- Détection de retard sur une tâche liée à l'objectif → signalement au prochain login
- Recommandation de la prochaine étape du parcours

3.8 Certificat vérifiable * (différenciant)

À la complétion d'un parcours (ou d'une étape majeure), génération automatique d'un certificat PDF avec :

- un numéro unique
- un QR code renvoyant vers une page de vérification (`/verify/{certificat_id}`)
- la liste des compétences/étapes validées

La page de vérification confirme l'authenticité sans exposer de données personnelles sensibles.

3.9 Recherche cross-données * (différenciant)

- Barre de recherche unique interrogeant en une requête : projets, tâches, documents, exercices – reliés par les clés étrangères du schéma
- Exemple : rechercher "JWT" retourne le document où c'est mentionné, l'exercice associé, et la tâche liée

3.10 Dashboard de progression

Tâches complétées, étapes du parcours terminées, prochaine révision programmée (répétition espacée), certificats obtenus.

3.11 Historique

Historique des conversations et recommandations.

3.12 Automatisation n8n

Rappels par email, notification avant échéance, rapport hebdomadaire de progression.

4 Modèle de données (schéma PostgreSQL)

Table	Champs principaux	Relations
Users	id, email, mot_de_passe (hashé), date_creation, rôle (user/admin)	1-1 avec Profiles
Profiles	id, user_id, préférences, ton_préfééré	appartient à Users
LearningProfile	id, user_id, objectif, niveau, disponibilité_hebdo, langue	appartient à Users
Roadmap	id, learning_profile_id, étapes (JSONB), date_generation	appartient à LearningProfile

Table	Champs principaux	Relations
Projects	id, user_id, nom, description, date_creation	1–N avec Tasks
Tasks	id, project_id, titre, priorité, statut, date_limite	appartient à Projects
Documents	id, user_id, nom_fichier, type, chemin_stockage, date_upload, statut	1–N avec DocumentChunks
DocumentChunks	id, document_id, contenu, embedding (vector)	appartient à Documents
Exercises	id, roadmap_step_id, question, type, réponse_attendue	appartient à Roadmap
ExerciseResults	id, exercise_id, user_id, réponse_donnée, score, prochaine_revision (date), feedback	appartient à Exercises et Users
Certificates	id, user_id, numéro_unique, qr_code_path, compétences_validées (JSONB), date_emission	appartient à Users
ChatHistory	id, user_id, message, réponse, date	appartient à Users
Notifications	id, user_id, type, contenu, statut_lu, date	appartient à Users

4.1 Champ clé pour la répétition espacée

`ExerciseResults.prochaine_revision` est calculé selon un algorithme simple type SM-2 (utilisé par Anki) :

- Bonne réponse → intervalle multiplié (ex. x2) avant la prochaine révision
- Mauvaise réponse → intervalle réinitialisé à court terme (ex. le lendemain)

Ce calcul est fait côté Django (fonction Python), stocké en base, puis exploité par n8n ou une tâche planifiée pour notifier l'utilisateur au bon moment.

4.2 Où et comment PostgreSQL est utilisé

PostgreSQL est la base centrale, connectée via l'ORM Django (aucune requête SQL brute nécessaire dans la majorité des cas).

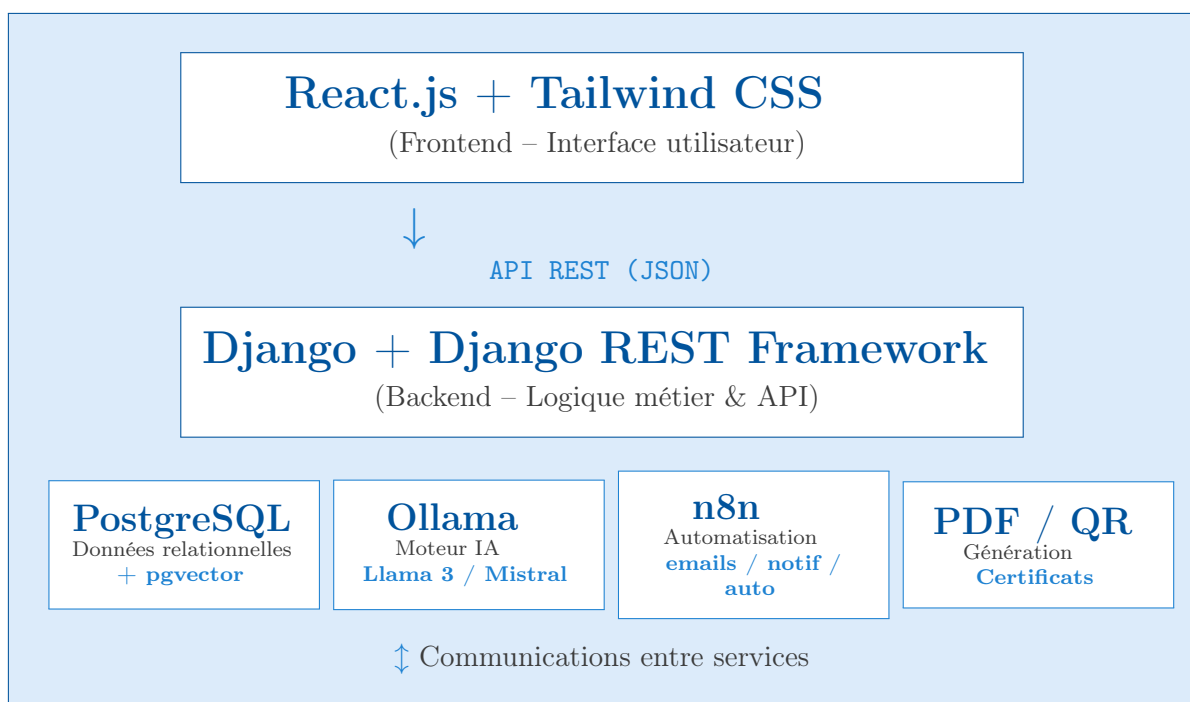
- Chaque modèle Django (`models.py`) correspond à une table, créée via les migrations (`makemigrations / migrate`)
- **pgvector** stocke les embeddings des documents (table `DocumentChunks`) pour la recherche sémantique du RAG – évite d'ajouter une base vectorielle séparée
- Les colonnes **JSONB** (`Roadmap.étapes`, `Certificates.compétences_validées`) stockent des structures flexibles générées par l'IA, dont la forme varie selon l'utilisateur
- La **recherche cross-données** (section 3.9) s'appuie sur des jointures SQL simples entre `Documents`, `Tasks`, `Exercises` via leurs clés étrangères communes (`user_id`, et éventuellement un tag/sujet partagé)

```
1 # settings.py
2 DATABASES = {
3     'default': {
4         'ENGINE': 'django.db.backends.postgresql',
5         'NAME': 'nova_ai_db',
6         'USER': 'nova_admin',
7         'PASSWORD': "à définir en variable d'environnement",
8         'HOST': 'localhost',
9         'PORT': '5432',
10    }
11 }
```

Listing 1 – Configuration PostgreSQL dans Django settings.py

Attention : identifiants en `.env`, jamais en dur dans le code, `.env` exclu du dépôt Git.

5 Architecture technique



6 Vision future (non développée dans le MVP – présentée comme roadmap en soutenance)

- Mode vocal (Text-to-Speech / Speech-to-Text, ex. Whisper)
- Simulation d’entretien d’embauche avec analyse des réponses
- Génération automatique de CV, portfolio, lettre de motivation
- Coach carrière : comparaison profil vs offre d’emploi avec plan de comblement automatique inséré dans le planning
- Migration technique possible vers LangChain (orchestration RAG), Qdrant (base vectorielle dédiée), Celery/Redis (tâches asynchrones lourdes), et choix multi-LLM (OpenAI, Gemini) si le projet est poursuivi au-delà du cadre académique

7 Exigences non fonctionnelles

7.1 Sécurité et confidentialité

- Mots de passe hashés
- Variables sensibles en ‘.env’, exclues du dépôt Git
- HTTPS/TLS en production
- Accès aux documents et certificats restreint au propriétaire (sauf page de vérification publique du certificat, qui n’expose aucune donnée personnelle)

7.2 Performance

- Temps de réponse cible : < 3 secondes pour une requête standard au LLM
- Prévoir un modèle léger (ex. Mistral 7B quantisé) si pas de GPU disponible

7.3 Scalabilité

Séparation claire frontend / backend / IA / automatisation / génération de documents.

8 Répartition de l’équipe (2 personnes)

Membre	Responsabilités
Membre 1	— Frontend React : Authentification, onboarding, Dashboard, interface documents/exercices, chat — Intégration IA : Ollama, chat, génération de roadmap/exercices, RAG — UI/UX Design : Maquettes, design system, expérience utilisateur

Membre	Responsabilités
Membre 2	— Backend Django : API REST, PostgreSQL, modèles de données, authentification — Base de données : Modélisation, migrations, pgvector, recherche cross-données — Automatisation & certificats : n8n, AI Mentor Engine, génération PDF/QR code, route de vérification — Algorithme : Répétition espacée, calcul des métriques de progression

8.1 Répartition détaillée des tâches

Module	Tâches	Responsable
3* Frontend	Authentification / Onboarding	Membre 1
	Dashboard & Progression	Membre 1
	Interface documents/exercices/chat	Membre 1
4* Backend	API REST & Authentification	Membre 2
	Modèles de données & PostgreSQL	Membre 2
	Recherche cross-données	Membre 2
	Algorithme répétition espacée	Membre 2
3* IA	Ollama / Chat / Génération roadmap	Membre 1
	Génération exercices / Correction	Membre 1

Module	Tâches	Responsable
	RAG & Analyse documents	Membre 1
3*Automatisation	n8n & AI Mentor Engine	Membre 2
	Emails / Rappels / Notifications	Membre 2
	Génération PDF/QR & Vérification	Membre 2

Organisation du travail

- **Méthodologie** Agile / Scrum – Sprints de 2 semaines
- **Outil de suivi** GitHub Projects / Trello
- **Communication** Réunions quotidiennes (15 min)
- **Revue de code** Chaque membre relit les PR de l'autre

9 Planning indicatif (8 semaines)

Semaine	Étape
1	Cadrage, modélisation PostgreSQL, setup environnements (React, Django, Ollama, n8n)
2	Authentification + onboarding (profil intelligent)
3	Dashboard + gestion des tâches/projets
4	Intégration Ollama – chat + génération de roadmap
5	Documents + RAG (upload, embeddings, bouton "Analyser")
6	Exercices générés + algorithme de répétition espacée
7	AI Mentor Engine (n8n) + génération de certificats (PDF/QR) + recherche cross-données

Semaine	Étape
8	Tests, corrections, dashboard de progression finalisé, préparation de la soutenance

10 Livrables attendus

1. Cahier des charges (ce document)
2. Schéma entité-relation PostgreSQL
3. Maquettes de l'interface (React + Tailwind)
4. Code source complet (frontend, backend, IA, automatisation, génération de certificats)
5. Documentation technique (installation, configuration, API)
6. Support de soutenance, incluant un tableau comparatif clair (section 1.3) et la section "Vision future"

11 Indicateurs de succès (KPIs) pour la soutenance

- Les 5 fonctionnalités différenciantes (section 1.3) sont toutes fonctionnelles, même en version simple
- Aucune fonctionnalité annoncée non implémentée n'est présentée comme acquise (la section 7 reste clairement labellisée "vision future")
- Le certificat généré est réellement vérifiable via son QR code pendant la démonstration live

12 Points ouverts à valider

- ☐ Modèle Llama 3 ou Mistral selon les ressources matérielles disponibles
- ☐ Format exact des exercices (QCM uniquement en MVP, ou aussi questions ouvertes avec correction textuelle)
- ☐ Hébergement pour la démo (local vs cloud léger type Render/Railway + Vercel)
- ☐ Bibliothèque de génération PDF (reportlab vs WeasyPrint) selon la complexité de mise en page souhaitée du certificat